

API Testing & REST Assured

Complete Interview Reference — v2.0

[REST](#) · [HTTP Concepts](#) · [Status Codes](#) · [Auth](#) · [REST Assured](#) · [Real Code](#) · [Senior Q&A](#)

50+

Interview Q&A

9

Sections

Code

Properly Formatted

SECTION 1 — API & REST Fundamentals

Q1. What is an API? Give a real-world analogy.

API (Application Programming Interface) is a contract between two software systems defining how they communicate.

Restaurant analogy: You (client) place an order → Waiter (API) carries it to the kitchen (server) → Waiter returns with the food (response).

Types relevant to SDET: REST · SOAP (legacy) · GraphQL · gRPC

Q2. What are the 6 REST architectural constraints?

1. **Uniform Interface** — Client & server agree on resource format (JSON/XML) and URI patterns.

2. **Stateless** — Each request is fully self-contained; server stores NO session.

3. **Cacheable** — Responses must declare cacheability to reduce server load.

4. **Client-Server** — UI and data storage are decoupled; evolve independently.

5. **Layered System** — Client unaware of load balancers, gateways in between.

6. **Code on Demand (optional)** — Server can send executable code to client.

■ **Senior Tip:** Statelessness enables horizontal scaling (add more servers). Cacheability enables CDNs. At senior level, know why each constraint exists architecturally.

Q3. REST vs SOAP — when would you choose each?

Feature	REST	SOAP
Protocol	HTTP	HTTP, SMTP, TCP
Format	JSON / XML / YAML	XML only
Speed	Faster (lightweight)	Slower (XML overhead)
Contract	OpenAPI / Swagger	WSDL (strict schema)
Security	HTTPS + OAuth 2.0	WS-Security (built-in)
Error handling	HTTP status codes	SOAP Fault element
Best for	Modern web / mobile APIs	Enterprise / banking / legacy

Q4. What is Idempotency? Which HTTP methods are idempotent?

Idempotent — calling the same operation N times produces identical result to calling it once.

Idempotent methods: GET · PUT · DELETE · HEAD · OPTIONS

NOT idempotent: POST (creates new resource each call) · PATCH (may differ)

Safe methods (read-only, zero side-effects): GET · HEAD · OPTIONS

■ **Senior Tip:** Idempotency matters for retry logic in distributed systems. Design automated retries only for idempotent methods.

■ SECTION 2 — HTTP Methods Deep Dive

■ GET

Retrieve resource. Safe + Idempotent + Cacheable. Params go in URL query string. No request body.

```
GET /api/users?page=1&size=10 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbGci...
```

■ POST

Create a new resource. NOT idempotent. Payload in body. Server determines the new resource URI.

```
POST /api/users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGci...
```

```
{
  "name": "Alice",
  "email": "alice@test.com",
  "role": "ADMIN"
}
```

■ PUT

Replace the ENTIRE resource. Idempotent. Client specifies URI. ALL fields must be sent.

```
PUT /api/users/5 HTTP/1.1
Content-Type: application/json
```

```
{
  "id": 5,
  "name": "Alice Updated",
  "email": "alice.new@test.com",
  "age": 31,
  "role": "ADMIN"
}
```

// If any field is omitted → that field becomes null/default

■ PATCH

Partial update — send ONLY the fields that changed. More efficient than PUT for large objects.

```
PATCH /api/users/5 HTTP/1.1
Content-Type: application/json
```

```
{
  "email": "patched@test.com"
}
```

// Only email changes. All other fields untouched.

■ DELETE

Remove a resource. Idempotent (deleting same resource twice = same result). Usually returns 204.

```
DELETE /api/users/5 HTTP/1.1
Authorization: Bearer eyJhbGci...
```

```
// Response: 204 No Content (resource deleted)
// Second call: 404 Not Found (already gone — still idempotent result)
```

■ OPTIONS

Return HTTP methods supported by the endpoint. Used for CORS preflight checks.

```
OPTIONS /api/users HTTP/1.1
Host: api.example.com

// Response Headers:
Allow: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Origin: https://myapp.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
```

■ HEAD

Like GET but returns ONLY headers — no response body. Check resource existence / get metadata.

```
HEAD /api/users/5 HTTP/1.1
Host: api.example.com

// Response (headers only, no body):
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 248
Last-Modified: Mon, 01 Apr 2024 10:00:00 GMT
```

■ **Senior Tip:** PUT vs PATCH is a very common 9+ year question. **PUT** = replace whole resource (client sends ALL fields). **PATCH** = send only what changed (efficient for large objects). Know which one your REST API design uses and why.

SECTION 3 — HTTP Status Codes

2xx SUCCESS

200 OK	Standard success. GET/PUT returns body. Most common success code.
201 Created	New resource created (POST/PUT). Location header has new resource URI.
202 Accepted	Request accepted but processing not done yet (async/queue operations).
204 No Content	Success but NO response body. Standard for DELETE, and PUT with no return.

3xx REDIRECTION

301 Moved Permanently	Resource has a new permanent URI. All future requests use new URI.
302 Found	Temporary redirect. Keep using original URI for future requests.
304 Not Modified	Cached version still valid. No data transferred — saves bandwidth.
307 Temporary Redirect	Like 302 but HTTP method must NOT change (POST stays POST).

4xx CLIENT ERRORS

400 Bad Request	Malformed syntax, invalid params, or deceptive routing. Fix your request.
401 Unauthorized	NOT authenticated. No valid credentials. Send correct auth token.
403 Forbidden	Authenticated but NOT authorized. User lacks required permission.
404 Not Found	Resource doesn't exist at this URI. Check the path.
405 Method Not Allowed	HTTP method not supported here. Response includes Allow header.
406 Not Acceptable	Server can't produce client's requested media type (Accept header).
409 Conflict	Duplicate resource, version conflict, or state conflict.
412 Precondition Failed	Conditional request header (If-Match) failed.
415 Unsupported Media Type	Server rejects the Content-Type sent by client.
422 Unprocessable Entity	Request syntax OK but semantic validation failed.
429 Too Many Requests	Rate limit exceeded. Check Retry-After response header.

5xx SERVER ERRORS

500 Internal Server Error	Generic server fault. Not client's fault. Safe to retry.
501 Not Implemented	Server doesn't support the requested functionality.
502 Bad Gateway	Upstream server returned invalid response (proxy issue).
503 Service Unavailable	Server down or overloaded. Temporary. Retry after delay.
504 Gateway Timeout	Upstream server timed out. Network or service is slow.

Q5. Difference between 401 and 403?

401 Unauthorized → Client is NOT authenticated. 'Who are you?' Credentials missing or wrong. Fix: send correct token / login first.

403 Forbidden → Client IS authenticated but lacks permission. 'I know who you are, but you can't do this.' Fix: grant the user the required role.

■ **Senior Tip:** Knowing 401 vs 403 saves debugging time. Always check: Is the user logged in? (401) vs Is the user's role correct? (403)

■ SECTION 4 — Authentication & Security

■ Basic Auth

Base64-encoded **username:password** in Authorization header. NOT encrypted — always use over HTTPS only.

```
// Request header (sent with every request):
Authorization: Basic am9objpzZWMyZXQ=

// Decode: Base64("john:secret") → am9objpzZWMyZXQ=

// In REST Assured:
given()
    .auth().basic("john", "secret")
    .when()
    .get("/secured-endpoint");
```

■ Bearer Token / JWT

Token sent in Authorization header. Stateless — server validates signature, no DB lookup needed.

```
// Request header:
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
    .eyJ1c2VySWQiOiIiIiwicm9leCI6MTcwMDAwMH0
    .SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

// JWT Structure: header.payload.signature
// Payload (Base64 decoded) contains:
{
  "userId": "5",
  "roles": ["ADMIN", "USER"],
  "exp": 1700000000, // expiry timestamp
  "iss": "auth.myapp.com"
}

// In REST Assured:
given()
    .header("Authorization", "Bearer " + jwtToken)
    .when()
    .get("/users");
```

■ OAuth 2.0 — Client Credentials Flow

Industry-standard delegated authorization. Get access token from Auth server, use for API calls.

```
// Step 1: Get token from Auth server
String token =
    given()
        .formParam("grant_type", "client_credentials")
        .formParam("client_id", "myClientId")
        .formParam("client_secret", "mySecret")
        .formParam("scope", "read:users write:orders")
    .when()
        .post("https://auth.example.com/oauth/token")
    .then()
        .statusCode(200)
        .extract()
        .path("access_token");

// Step 2: Use token in API calls
given()
    .header("Authorization", "Bearer " + token)
    .when()
    .get("https://api.example.com/users");
```

■ API Key Auth

Simple secret key in header or query param. Easy to implement but less secure — rotate keys regularly.

```
// Preferred: API key in header (not visible in server logs)
given()
  .header("X-API-Key", "your-secret-api-key-here")
  .when()
  .get("/data");

// Less secure: API key in query param (appears in logs/URL)
given()
  .queryParams("api_key", "your-secret-api-key-here")
  .when()
  .get("/data");
```

Q6. OAuth 1.0 vs OAuth 2.0 — key differences?

Aspect	OAuth 1.0	OAuth 2.0
Security	Cryptographic signatures	Bearer tokens + HTTPS mandatory
Complexity	Complex — signature required per call	Simple — send token as header
Token Types	Request token + Access token	Access token + Refresh token
Flexibility	Web apps primarily	Web, mobile, server-to-server
Grant Types	Single authorization flow	Auth Code, Client Creds, Implicit...
Used today	Legacy / niche systems	Modern standard — use this ✓

SECTION 5 — REST Assured Core

Maven Dependency (pom.xml)

```
<!-- REST Assured core -->
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>5.4.0</version>
  <scope>test</scope>
</dependency>

<!-- JSON Schema Validator -->
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>json-schema-validator</artifactId>
  <version>5.4.0</version>
  <scope>test</scope>
</dependency>

<!-- Jackson for POJO serialization/deserialization -->
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.15.2</version>
</dependency>
```

The given() / when() / then() Pattern

```
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;

given() // ① REQUEST SPEC – set up request
  .baseUrl("https://api.example.com")
  .header("Content-Type", "application/json")
  .header("Authorization", "Bearer " + token)
  .header("Accept", "application/json")
  .queryParams("page", 1)
  .queryParams("size", 10)
  .log().all() // log full request to console

.when() // ② ACTION – HTTP method + path
  .get("/users")

.then() // ③ RESPONSE VALIDATION
  .log().all() // log full response
  .statusCode(200)
  .contentType("application/json")
  .header("X-Rate-Limit-Remaining", notNullValue())
  .body("data.size()", greaterThan(0))
  .body("data[0].name", notNullValue())
  .body("data[0].email", containsString("@"))
  .body("total", greaterThan(0))
  .time(lessThan(3000L)); // must respond within 3 seconds
```

■ **Senior Tip:** given() = request setup, when() = HTTP action, then() = validation. This BDD-style DSL makes tests self-documenting.

All HTTP Methods in REST Assured

SECTION 6 — REST Assured Advanced Techniques

RequestSpecification — Reusable Base Setup

[illegible]

Response Extraction & JsonPath

[illegible]

JSON Schema Validation


```

// Basic Auth (sends every request)
given()
    .auth().basic("username", "password")
    .when().get("/secured");

// Preemptive Basic Auth (skips 401 challenge round-trip – faster)
given()
    .auth().preemptive().basic("username", "password")
    .when().get("/secured");

// Bearer Token / JWT
given()
    .header("Authorization", "Bearer " + jwtToken)
    .when().get("/users");

// OAuth 2.0 Bearer
given()
    .auth().oauth2(accessToken)
    .when().get("/users");

// API Key in header
given()
    .header("X-API-Key", apiKey)
    .when().get("/data");

// Digest Authentication
given()
    .auth().digest("username", "password")
    .when().get("/digest-secured");

// Disable SSL validation (test environments only – NEVER in production)
given()
    .relaxedHTTPSValidation()
    .when().get("https://self-signed-cert-server/api");

```

Query Params · Path Params · Headers · Cookies

```
// ■■■ Query Parameters → appended to URL
// Result: GET /users?page=1&size=10&sort=name,asc&filter=ACTIVE
given()
    .queryParams("page", 1)
    .queryParams("size", 10)
    .queryParams("sort", "name,asc")
    .queryParams("filter", "ACTIVE")
    .when().get("/users");

// ■■■ Path Parameters → embedded in URL
// Result: GET /users/5/orders/12
given()
    .pathParam("userId", 5)
    .pathParam("orderId", 12)
    .when().get("/users/{userId}/orders/{orderId}");

// ■■■ Multiple Headers
import io.restassured.http.Header;
import io.restassured.http.Headers;

Header h1 = new Header("Content-Type", "application/json");
Header h2 = new Header("Authorization", "Bearer " + token);
Header h3 = new Header("X-Request-ID", UUID.randomUUID().toString());
Headers headers = new Headers(h1, h2, h3);

given().headers(headers).when().get("/users");

// ■■■ Cookies
given()
    .cookie("session_id", "abc123xyz")
    .cookie("preference", "dark-mode")
    .when().get("/dashboard");

// ■■■ Form Data (application/x-www-form-urlencoded)
given()
    .contentType("application/x-www-form-urlencoded")
    .formParam("username", "admin")
    .formParam("password", "pass123")
    .when().post("/login");

// ■■■ File Upload (multipart)
given()
    .multiPart("file", new File("src/test/resources/test-data.csv"))
    .multiPart("description", "Test file upload")
    .when().post("/upload")
    .then().statusCode(200);
```

Request & Response Logging

```

// Log everything – request + response (best for debugging)
given()
    .log().all()
.when()
    .get("/users")
.then()
    .log().all()
    .statusCode(200);

// Log ONLY on failure (production-friendly – less noise in passing tests)
given()
.when()
    .get("/users")
.then()
    .log().ifValidationFails()
    .statusCode(200);

// Log specific parts of request
given()
    .log().headers()    // only request headers
    .log().params()     // only query/form params
    .log().body()       // only request body
    .when().post("/users");

// Log specific parts of response
.then()
    .log().status()     // only status line
    .log().headers()    // only response headers
    .log().body()       // only response body
    .statusCode(200);

```

Timeout Configuration

Soft Assertions — Continue on Failure

```

import org.assertj.core.api.SoftAssertions;

Response response =
    given(requestSpec)
        .when()
            .get("/users/1")
        .then()
            .extract()
            .response();

// SoftAssertions – collect ALL failures, report together at end
SoftAssertions softly = new SoftAssertions();

softly.assertThat(response.getStatusCode())
    .as("Status code")
    .isEqualTo(200);

softly.assertThat(response.jsonPath().getString("name"))
    .as("User name")
    .isEqualTo("Alice");

softly.assertThat(response.jsonPath().getInt("age"))
    .as("User age")
    .isGreaterThan(18);

softly.assertThat(response.getHeader("Content-Type"))
    .as("Content-Type header")
    .contains("application/json");

softly.assertThat(response.getTime())
    .as("Response time (ms)")
    .isLessThan(2000L);

softly.assertAll();    // ← throws ONE error listing ALL failures together

```

■ **Senior Tip:** Use `SoftAssertions` when you want to validate ALL response fields in a single test run. Stops you re-running tests just to find the next failure.

SECTION 7 — Serialization, Deserialization & E2E Flow

POJO Class (with Lombok)

```
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;

@Data // generates getters, setters, toString, equals, hashCode
@Builder // enables User.builder().name("Alice").build()
@NoArgsConstructor // required by Jackson for deserialization
@AllArgsConstructor // required by @Builder
@JsonIgnoreProperties(ignoreUnknown = true) // ignore extra fields in JSON
public class User {
    private Integer id;
    private String name;
    private String email;
    private String role;
    private Integer age;
    private Boolean active;
}
```

Serialization: Java Object → JSON (sending data)

```
// Build POJO using Builder pattern
User newUser = User.builder()
    .name("Alice")
    .email("alice@test.com")
    .role("ADMIN")
    .age(30)
    .build();

// REST Assured + Jackson serializes POJO → JSON automatically
given()
    .contentType(ContentType.JSON)
    .body(newUser) // Jackson converts: {"name":"Alice","email":"alice@test.com",...}
.when()
    .post("/api/users")
.then()
    .statusCode(201)
    .body("name", equalTo("Alice"))
    .body("email", equalTo("alice@test.com"))
    .body("id", notNullValue());
```

Deserialization: JSON Response → Java Object (receiving data)

```

// Deserialize single object
User user =
    given()
    .when()
    .get("/api/users/1")
    .then()
    .statusCode(200)
    .extract()
    .as(User.class);    // Jackson converts JSON → User POJO

System.out.println(user.getName());    // Alice
System.out.println(user.getEmail());   // alice@test.com
System.out.println(user.getAge());     // 30

// Deserialize list of objects
List<User> users =
    given()
    .when()
    .get("/api/users")
    .then()
    .statusCode(200)
    .extract()
    .body()
    .jsonPath()
    .getList("data", User.class);

users.forEach(u -> System.out.println(u.getName()));

```

Complete E2E API Test — Part 1: Auth + GET

```

public class E2EBookstoreTest {

    private static String token;
    private static String userId = "9b5f49ab-eea9-45f4-9d66-bcf56a531b85";
    private static String bookIsbn;

    @BeforeClass
    public static void authenticate() {
        RestAssured.baseURI = "https://bookstore.toolsqa.com";

        // Step 1 – Get Bearer token
        Response tokenResp =
            given()
                .contentType("application/json")
                .body("{\"userName\":\"TOOLSQA-Test\",\"password\":\"Test@@123\"}")
            .when()
                .post("/Account/v1/GenerateToken");

        Assert.assertEquals(tokenResp.getStatusCode(), 200);
        token = tokenResp.jsonPath().getString("token");
        Assert.assertNotNull(token, "Token must not be null");
    }

    @Test(priority = 1)
    public void getAllBooks() {
        // Step 2 – Get books (no auth needed)
        Response resp =
            given()
                .when()
                    .get("/BookStore/v1/Books")
                .then()
                    .statusCode(200)
                    .body("books.size()", greaterThan(0))
                    .extract().response();

        List<Map<String, String>> books = resp.jsonPath().getList("books");
        bookIsbn = books.get(0).get("isbn");
        System.out.println("Using book ISBN: " + bookIsbn);
    }
}

```

Complete E2E API Test — Part 2: POST + DELETE + Verify

```

@Test(priority = 2, dependsOnMethods = "getAllBooks")
public void addBookToUser() {
    // Step 3 – Add book (auth required)
    String body = String.format(
        "{ \"userId\": \"%s\", \"collectionOfIsbns\": [{\"isbn\": \"%s\"}] }",
        userId, bookIsbn
    );

    given()
        .header("Authorization", "Bearer " + token)
        .contentType("application/json")
        .body(body)
    .when()
        .post("/BookStore/v1/Books")
    .then()
        .statusCode(201);
}

@Test(priority = 3, dependsOnMethods = "addBookToUser")
public void deleteBookAndVerify() {
    // Step 4 – Delete the book
    String deleteBody = String.format(
        "{ \"isbn\": \"%s\", \"userId\": \"%s\" }",
        bookIsbn, userId
    );

    given()
        .header("Authorization", "Bearer " + token)
        .contentType("application/json")
        .body(deleteBody)
    .when()
        .delete("/BookStore/v1/Book")
    .then()
        .statusCode(204);

    // Step 5 – Verify bookshelf is now empty
    Response userResp =
        given()
            .header("Authorization", "Bearer " + token)
        .when()
            .get("/Account/v1/User/" + userId)
        .then()
            .statusCode(200)
            .extract().response();

    List<Map<String, String>> userBooks = userResp.jsonPath().getList("books");
    Assert.assertEquals(userBooks.size(), 0, "Bookshelf must be empty");
}
}

```

■ SECTION 8 — Senior-Level Interview Q&A; (9+ Years)

Q7. How did you design your API automation framework from scratch?

Components I implement:

- **BaseApiTest** — RequestSpecification, token refresh, baseURI per environment
- **API Client layer** — UserApiClient, OrderApiClient (one class per service)
- **POJO/DTO layer** — Java objects with Lombok for request/response
- **Schema layer** — JSON schemas in /resources/schemas/ for contract validation
- **TestNG/JUnit** — parallel execution, groups (smoke/regression), retry listener
- **Allure reporting** — request/response attached to each test step
- **CI/CD** — Jenkins/GitHub Actions pipeline runs on PR and nightly
- **Config layer** — property files per environment (DEV / QA / STAGE / PROD)

■ **Senior Tip:** Be specific about WHY you made each decision — why RequestSpecBuilder, how you handle token expiry, why TestNG over JUnit.

Q8. How do you handle token expiry in long-running test suites?

TokenManager utility class:

- Token cached in static variable with parsed expiry (exp claim from JWT)
- Before each request: check if token expires within next 60 seconds
- If about to expire: silently re-authenticate and update cached token
- Thread-safe for parallel execution: use synchronized block or AtomicReference
- Used via: TokenManager.getValidToken() — caller never worries about expiry

■ **Senior Tip:** This shows you think about production-like test reliability, not just happy-path.

Q9. How do you handle flaky API tests?

Root causes and fixes:

- **Async operations** — API returns 202, then processes in background.

Fix: use Awaitility to poll until status = COMPLETED

- **Shared test data** — tests interfere with each other.

Fix: each test creates and owns its own data, deletes in @AfterMethod

- **Eventual consistency** — microservice data sync delay.

Fix: retry assertion with backoff using Awaitility

- **Rate limiting** — too many test calls trigger 429.

Fix: add delay between requests, use separate test API key

- **Retry strategy** — TestNG RetryAnalyzer (max 2 retries) + Slack alert on retry

■ **Senior Tip:** Awaitility example: Awaitility.await().atMost(10, SECONDS).pollInterval(1, SECONDS).until(() -> getOrderStatus(id).equals("SHIPPED"));

Q10. What is contract testing and have you used it?

Contract testing (Pact):

- Consumer writes a Pact (JSON contract) defining what it expects from the provider
- Provider runs Pact verification against its actual implementation
- Contract stored in Pact Broker — visible to both teams
- No need to spin up both services together

Key benefit: Catches breaking changes before integration tests.

Enables independent service deployment in microservices architecture.

■ **Senior Tip:** Pact testing is a major differentiator at senior SDET level. If not used, study the concept — interviewers increasingly ask about it.

Q11. How do you integrate API tests in CI/CD pipeline?

My typical pipeline setup:

- **On PR** — Smoke tests only (20-30 critical tests, < 5 min)
- **On merge to main** — Full regression (parallel, 4 threads, ~30 min)
- **Nightly** — Full suite + performance SLA checks against STAGE
- **Reporting** — Allure report published; Slack webhook on failure
- **Gate** — Build fails if failure rate > 5%; retry flaky tests once

Maven command: `mvn test -Denv=qa -Dgroups=smoke -Dthreads=4`

■ **Senior Tip:** Show pipeline efficiency thinking: test grouping, parallel execution, and how you balance speed vs. coverage.

Q12. How do you perform negative/boundary testing on APIs?

Scenarios I always cover:

- Missing required fields → expect 400 Bad Request
- Invalid data types (string where int expected) → expect 400
- Field length exceeds maximum → expect 400 or 422
- Expired or tampered JWT token → expect 401
- Valid token but wrong role → expect 403
- Non-existent resource ID → expect 404
- Wrong HTTP method on endpoint → expect 405
- Wrong Content-Type header → expect 415
- Rate limit exhaustion (send N+1 requests) → expect 429
- SQL injection / XSS in params → expect 400, no execution
- Empty body on POST → expect 400 with clear error message

■ **Senior Tip:** Negative testing finds the most critical bugs. Always assert error response body — good APIs return actionable messages, not just codes.

■ SECTION 9 — Quick Reference Cheat Sheet

Hamcrest Matchers — Complete Reference

[illegible]

HTTP Status Code Quick Recall

200	OK — success with body
201	Created — POST/PUT created resource
204	No Content — success, no body (DELETE/PUT)
301	Moved Permanently — update your bookmarks
304	Not Modified — use your cached copy
400	Bad Request — fix your request
401	Unauthorized — authenticate first

403	Forbidden — logged in but not allowed
404	Not Found — resource doesn't exist
405	Method Not Allowed — wrong HTTP verb
409	Conflict — duplicate / version mismatch
415	Unsupported Media Type — wrong Content-Type
422	Unprocessable Entity — validation failed
429	Too Many Requests — rate limited
500	Internal Server Error — server bug
502	Bad Gateway — upstream returned garbage
503	Service Unavailable — server down/overloaded
504	Gateway Timeout — upstream timed out

Key Interview One-Liners

```

PUT vs POST → PUT = replace at known URI (idempotent);
               POST = create, server chooses new URI (not idempotent)

PUT vs PATCH → PUT = replace ALL fields;
               PATCH = update ONLY the specified fields

401 vs 403 → 401 = not authenticated; 403 = authenticated but forbidden

given/when/then → given = request setup; when = HTTP method;
                  then = response validation

Header vs Headers → Header = single name-value pair;
                   Headers = collection of Header objects

Serialization → Java Object → JSON (sending data in request body)
Deserialization → JSON → Java Object (processing response body)

JsonPath → Path expression to extract values from JSON
          (like XPath for XML: data[0].name, total)

Schema validation → validates structure & types, not just values

Idempotent → same result regardless of how many times called
            (GET, PUT, DELETE are idempotent; POST is not)

Stateless REST → each HTTP request is independent;
                server stores NO session between calls

relaxedHTTPSValidation() → disables SSL cert check;
                          USE ONLY in test env, NEVER production

```

■ ■ Top 7 mistakes in API automation:

- 1) No JSON schema validation (misses structure changes)
- 2) Tests depend on each other (ordering dependency = fragile suite)
- 3) No token refresh logic (long suites fail mid-run)
- 4) Hardcoded test data in code (not maintainable)
- 5) No negative test cases (misses critical error-path bugs)
- 6) No performance SLA checks (performance regressions go unnoticed)
- 7) Logging disabled in CI (impossible to debug failures)

■ **Senior Tip: Senior differentiators (9+ years):** Contract testing with Pact · API gateway testing · Microservices E2E testing strategy · Performance SLA enforcement in CI/CD · GraphQL testing · Mentoring juniors on framework design · Shift-left API testing

integrated with dev workflow.